

INTRODUCTION TO PYTHON FUNCTION



Mr. Rajendra Singh Rawat
Fellow, Indian Institute of Geomagnetism,
Colaba Geomagnetic Research Laboratory (CGRL)
<https://sites.google.com/view/rajendrasrawat>

INTRODUCTION TO PYTHON FUNCTION

Function:

Functions are one of the fundamental building blocks in Python programming language.

Functions are used when you have a block of statements that needs to be executed multiple times within the program.

Rather than writing the block of statements repeatedly to perform the action, you can use a function to perform that action.

This block of statements are grouped together and is given a name which can be used to invoke it from other parts of the program.

You write a function once but can execute it any number of times you like.

Functions also reduce the size of the program by eliminating rudimentary code.

Functions can be either Built-in Functions or User-defined functions.

The Python interpreter has a number of functions that are built into it and are always available.

INTRODUCTION TO PYTHON FUNCTION

Let's discuss few more built-in functions.:

For example,

```
1. >>> abs(-3)
3
```

```
2. >>> min(1, 2, 3, 4, 5)
1
```

```
3. >>> max(4, 5, 6, 7, 8)
8
```

```
4. >>> divmod(5, 2)
(2, 1)
```

```
5. >>> divmod(8.5, 3)
(2.0, 2.5)
```

```
6. >>> pow(3, 2)
9
```

```
7. >>> len("Japan")
5
```

A Few Built-in Functions in Python		
Function Name	Syntax	Explanation
abs()	<i>abs(x)</i> where x is an integer or floating-point number.	The abs() function returns the absolute value of a number.
min()	<i>min(arg_1, arg_2, arg_3,..., arg_n)</i> where arg_1, arg_2, arg_3 are the arguments.	The min() function returns the smallest of two or more arguments.
max()	<i>max(arg_1, arg_2, arg_3,...,arg_n)</i> where arg_1, arg_2, arg_3 are the arguments.	The max() function returns the largest of two or more arguments.
divmod()	<i>divmod(a, b)</i> where a and b are numbers representing numerator and denominator.	The divmod() function takes two numbers as arguments and return a pair of numbers consisting of their quotient and remainder. For example, if a and b are integer values, then the result is the same as (a // b, a % b). If either a or b is a floating-point number, then the result is (q, a % b), where q is the whole number of the quotient.
pow()	<i>pow(x, y)</i> where x and y are numbers.	The pow(x, y) function returns x to the power y which is equivalent to using the power operator: x**y.
len()	<i>len(s)</i> where s may be a string, byte, list, tuple, range, dictionary or a set.	The len() function returns the length or the number of items in an object.

INTRODUCTION TO PYTHON FUNCTION

Commonly Used Modules:

Modules in Python are reusable libraries of code having .py extension, which implements a group of methods and statements.

Python comes with many built-in modules as part of the standard library.

To use a module in your program, import the module using import statement.

All the import statements are placed at the beginning of the program.

The syntax for import statement is,
`import module_name`

For example, you can import the math module as
1. `>>>import math`

The math module is part of the Python standard library which provides access to various mathematical functions and is always available to the programmer.

INTRODUCTION TO PYTHON FUNCTION

The syntax for using a function defined in a module is,

`module_name.function_name()`

The module name and function name are separated by a dot.

Here we list some of the functions supported by math module.

1. `>>> import math`

2. `>>> print(math.ceil(5.4))`
6

3. `>>> print(math.sqrt(4))`
2.0

4. `>>> print(math.pi)`
3.141592653589793

5. `>>> print(math.cos(1))`
0.5403023058681398

6. `>>> math.factorial(6)`
720

7. `>>> math.pow(2, 3)`
8.0

INTRODUCTION TO PYTHON FUNCTION

The built-in function `dir()` returns a sorted list of comma separated strings containing the names of functions, classes and variables as defined in the module.

For example,

you can find all the functions supported by the `math` module by passing the module name as an argument to the `dir()` function.

1. `>>> dir(math)`

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign', 'cos', 'cosh', 'degrees', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'pi', 'pow', 'radians', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc']
```

Another useful module in the Python standard library is the `random` module which generates random numbers.

1. `>>> import random`

2. `>>> print(random.random())`

```
0.2551941897892144
```

3. `>>> print(random.randint(5,10))`

```
9
```


INTRODUCTION TO PYTHON FUNCTION

User-defined functions Definition:

You can create your own functions and use them as and where it is needed.

User-defined functions are reusable code blocks created by users to perform some specific task in the program.

The syntax for function definition is,

```
def function_name(parameter_1, parameter_2, ..., parameter_n):  
    statement(s)  
    return [expression_list]
```

INTRODUCTION TO PYTHON FUNCTION

In Python, a function definition consists of the `def` keyword, followed by

1. The name of the function. The function's name has to adhere to the same naming rules as variables: use letters, numbers, or an underscore, but the name cannot start with a number.

Also, you cannot use a keyword as a function name.

2. A list of parameters to the function are enclosed in parentheses and separated by commas.

Some functions do not have any parameters at all while others may have one or more parameters.

3. A colon is required at the end of the function header.

The first line of the function definition which includes the name of the function is called the function header.

4. Block of statements that define the body of the function start at the next line of the function header and they must have the same indentation level.

INTRODUCTION TO PYTHON FUNCTION

Flow of Execution:

- The order in which statements are executed is called the flow of execution.
- Execution always begins at the first statement of the program.
- Statements are executed one at a time, in order, from top to bottom.
- Function definitions do not alter the flow of execution of the program, but remember that statements inside the function are not executed until the function is called.
- Function calls are like a bypass in the flow of execution.
Instead of going to the next statement, the flow jumps to the first line of the called function, executes all the statements there, and then comes back to pick up where it left off.

INTRODUCTION TO PYTHON FUNCTION

Note:

When you read a program, don't read from top to bottom.

Instead, follow the flow of execution.

This means that you will read the def statements as you are scanning from top to bottom, but you should skip the statements of the function definition until you reach a point where that function is called.

For example:

```
def add_numbers(x,y):  
    sum = x + y  
    return sum
```

```
print("The sum is", add_numbers(5, 20))
```


INTRODUCTION TO PYTHON FUNCTION

Number of Arguments:

By default, a function must be called with the correct number of arguments.

If your function expects 2 arguments, you must call it with exactly 2 arguments.

Example:

This function expects 2 arguments, and gets 2 arguments::

```
def my_function(fname, lname):  
    print(fname + " " + lname)
```

```
my_function("Emil", "Refsnes")
```

INTRODUCTION TO PYTHON FUNCTION

Default Parameter Values:

You can assign default values to parameters.

If the function is called without an argument, it uses the default value:

Example

```
def my_function(name = "friend"):
    print("Hello", name)
```

```
my_function("Emil")
my_function("Tobias")
my_function()
my_function("Linus")
```


INTRODUCTION TO PYTHON FUNCTION

Keyword Arguments:

You can send arguments with the key = value syntax.

Example:

```
def my_function(animal, name):  
    print("I have a", animal)  
    print("My", animal + "'s name is", name)  
  
my_function(animal = "dog", name = "Buddy")
```

‘This way, with keyword arguments, the order of the arguments does not matter.

Example:

```
def my_function(animal, name):  
    print("I have a", animal)  
    print("My", animal + "'s name is", name)  
  
my_function(name = "Buddy", animal = "dog")
```

INTRODUCTION TO PYTHON FUNCTION

Scope:

A variable is only available from inside the region it is created. This is called scope.

Local Scope:

A variable created inside a function belongs to the local scope of that function, and can only be used inside that function.

Example:

```
def myfunc():  
    x = 50  
    print(x)  
  
myfunc()
```


INTRODUCTION TO PYTHON FUNCTION

Function Inside Function:

As explained in the example above, the variable x is not available outside the function, but it is available for any function inside the function:

Example:

```
def myfunc():  
    x = 70  
    def myinnerfunc():  
        print(x)  
    myinnerfunc()  
  
myfunc()
```

INTRODUCTION TO PYTHON FUNCTION

Global Scope:

A variable created in the main body of the Python code is a global variable and belongs to the global scope.

Global variables are available from within any scope, global and local.

Example:

```
x = 80
```

```
def myfunc():  
    print(x)
```

```
myfunc()
```

```
print(x)
```


INTRODUCTION TO PYTHON FUNCTION

Lambda Functions:

A lambda function is a small anonymous function.

A lambda function can take any number of arguments, but can only have one expression.

Syntax:

lambda arguments : expression

The expression is executed and the result is returned:

Example:

Add 10 to argument a, and return the result:

```
x = lambda a : a + 10  
print(x(5))
```

Example:

Multiply argument a with argument b and return the result:

```
x = lambda a, b : a * b  
print(x(5, 6))
```

INTRODUCTION TO PYTHON FUNCTION

Example:

Summarize argument a, b, and c and return the result:

```
x = lambda a, b, c : a + b + c  
print(x(5, 6, 2))
```

For Example:

divisibility by 2 and returns "Even" or "Odd" accordingly.

```
check = lambda x: "Even" if x % 2 == 0 else "Odd"  
print(check(4))  
print(check(7))
```


INTRODUCTION TO PYTHON FUNCTION

Lambda with Built-in Functions:

Lambda functions are commonly used with built-in functions like `map()`, `filter()`, and `sorted()`.

Using Lambda with `map()`:

The `map()` function applies a function to every item in an iterable:

Example:

Double all numbers in a list:

```
numbers = [1, 2, 3, 4, 5]
doubled = list(map(lambda x: x * 2, numbers))
print(doubled)
```

Using Lambda with `filter()`:

The `filter()` function creates a list of items for which a function returns `True`:

Example:

Filter out odd numbers from a list:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
odd_numbers = list(filter(lambda x: x % 2 != 0, numbers))
print(odd_numbers)
```

INTRODUCTION TO PYTHON FUNCTION

Using Lambda with sorted():

The sorted() function can use a lambda as a key for custom sorting:

Example

Sort strings by length:

```
words = ["apple", "pie", "banana", "cherry"]  
sorted_words = sorted(words, key=lambda x: len(x))  
print(sorted_words)
```


INTRODUCTION TO PYTHON FUNCTION

Recursion:

Recursion is a programming technique where a function calls itself either directly or indirectly to solve a problem by breaking it into smaller, simpler subproblems.

Recursion is especially useful for problems that can be divided into identical smaller tasks, such as mathematical calculations, tree traversals or divide-and-conquer algorithms.

Working of Recursion:

A recursive function is just like any other function except that it calls itself in its body.

Let's see basic structure of recursive function:

```
def recursive_function(parameters):  
    if base_case_condition:  
        return base_result  
    else:  
        return recursive_function(modified_parameters)
```

INTRODUCTION TO PYTHON FUNCTION

Example:

A simple recursive function that counts down from 5:

```
def countdown(n):  
    if n <= 0:  
        print("Done!")  
    else:  
        print(n)  
        countdown(n - 1)
```

```
countdown(5)
```


INTRODUCTION TO PYTHON FUNCTION

Example:

Identifying base case and recursive case:

```
def factorial(n):  
    # Base case  
    if n == 0 or n == 1:  
        return 1  
    # Recursive case  
    else:  
        return n * factorial(n - 1)  
  
print(factorial(5))
```